

DevEx with Developer Hub

Version 1, 2026-07-30

DevEx with Developer Hub

Introduction

Welcome to this quick course on the *DevEx with Developer Hub*. This course is the **fourth** in a series of **six** courses about **Application Platform Delivery**:

- [*Application Platform Delivery Foundations*](#)
- [*Software Factory & Developer Experience*](#)
- [*CICD*](#)
- *DevEx with Developer Hub(This course)*
- [*Application Modernization*](#)
- [*Trusted Software Supply Chain \(TSSC\)*](#)

What's in this course?

This training provides a comprehensive understanding of a modern Software Factory and its role in automating the software delivery lifecycle. Participants will learn how supporting services, Continuous Integration (CI), OpenShift Pipelines (Tekton), Continuous Delivery (CD), and promotion pipelines work together to build, validate, deploy, and promote applications across environments. By the end of the training, learners will understand how to implement standardized, automated, and GitOps-driven software delivery workflows on Red Hat OpenShift, enabling faster, more secure, and reliable application delivery.

Duration: 1.5 hours

Contributors

The PTL team acknowledges the valuable contributions of the following Red Hat associates:

- .. (SME)
- .. (SME)
- Yogita Soni (Content Architect)
- Anna Swicegood (Learning Product Manager)

Objectives

On completing this course, you should be able to:

Based on the section index, here are concise learning objectives:

Learning Objectives

By the end of this module, you will be able to:

- Integrate Dev Spaces with GitLab to enable source code management and developer workflows.
- Create and use Dev Spaces workspaces for cloud-native application development.
- Understand how Continuous Integration (CI) pipelines are triggered and executed from developer workspaces.
- Deploy applications to OpenShift using automated delivery workflows.
- Explain the role of promotion pipelines in advancing applications across development, testing, and production environments.

Prerequisites

This course assumes that you have the following prior experience:

1. [Red Hat OpenShift Container Platform \(OCP\) Technical Overview](#) or Kubernetes Basics

Lab Environment

Click the [RHDP](#) link above, log in to RHDP, and click **Order** to request a new [Service Enablement: App Platform Software Factory](#). On the order page for the catalog item, do the following:

- **Activity:** Select **Practice/Enablement**
- **Purpose:** Select **Learning about the Product**
- **Salesforce ID:** If you have an opportunity ID, enter it into this field, otherwise select **Project**, and then enter "Learning about RHCL" in the text field. You will see a warning message that a Salesforce ID is required, but you can safely ignore it.
- **Features:** Select **Enable Let's Encrypt**
- **Region:** Select the AWS region closest to your location
- **User count:** **1**
- **Control Plane Count:** **1**
- **OpenShift Version:** **4.16**
- **Control Plane Instance Type:** **m6a.4xlarge**
- Select the checkbox at the bottom of the page to confirm that lab costs will be charged to your cost center, and click **Order**.



The lab environment will take an hour to provision. You can check the status and details of the OpenShift cluster from the **Services** page of RHDP. The classroom is available for 48 hours initially. You can extend the duration of classroom availability by clicking **Help > Get Technical Help** in RHDP, and then requesting an extension by opening a ServiceNow ticket with the RHDP team.

RHDP Portal Links

- [RedHat Associates](#)
- [RedHat Partners](#)

Developer Hub and the IDP

Developer Experience Matters

Why Developer Experience Matters

- Modern software development requires developers to work with numerous tools throughout the Software Development Lifecycle (SDLC).
- A typical developer interacts with:
 - Source code repositories
 - Development environments
 - CI/CD pipelines
 - GitOps deployment tools
 - Container registries
 - Documentation portals
 - Security scanners
 - Monitoring and observability platforms



- Each tool solves a specific problem. However, switching between multiple tools increases the amount of information developers must understand before they can deliver software.
- This additional mental effort is commonly referred to as **developer cognitive load**.
- Platform Engineering aims to reduce this cognitive load by providing standardized workflows and self-service capabilities, allowing developers to spend more time writing applications and less time managing platform complexity.



Great platforms are not measured by the number of tools they provide, but by how simple they make software delivery for developers.

What is Developer Experience?



- *Developer Experience (DevEx)* describes the overall experience developers have while building, testing, deploying, and operating software.
- It encompasses the tools, workflows, platforms, environments, and organizational practices that influence how efficiently developers can deliver applications.

Developer Experience is the user experience developers have while interacting with a platform, its tools, workflows, and supporting services.

- The primary goals of DevEx are to:
 - Reduce friction during software development.
 - Improve developer productivity.
 - Standardize development workflows.
 - Enable developers to focus on delivering business value.

Benefits of Improving Developer Experience

Improving Developer Experience benefits both developers and organizations.

Technical Benefits

- Reduced cognitive load
- Standardized development workflows
- Faster onboarding of new developers
- Faster software delivery
- Improved software quality
- Consistent engineering practices

Business Benefits

- Higher developer productivity

- Faster time to market
- Lower software development costs
- Reduced developer burnout
- Higher employee retention
- Increased innovation



Organizations that invest in Developer Experience often observe improvements in software delivery performance, developer satisfaction, and overall business outcomes.

Developer Experience and Platform Engineering

Developer Experience is closely related to several Platform Engineering concepts.

Platform Engineering

Builds, operates, and maintains the internal platform that enables developers to build and deliver software efficiently.

Internal Developer Platform (IDP)

A collection of standardized tools, automation, services, and workflows that developers use throughout the Software Development Lifecycle.

Developer Experience (DevEx)

Measures how effectively developers can use the Internal Developer Platform to deliver software.

Internal Developer Portal

A centralized, self-service interface where developers discover services, documentation, templates, and workflows required to build and operate applications.

Introducing Red Hat Developer Hub

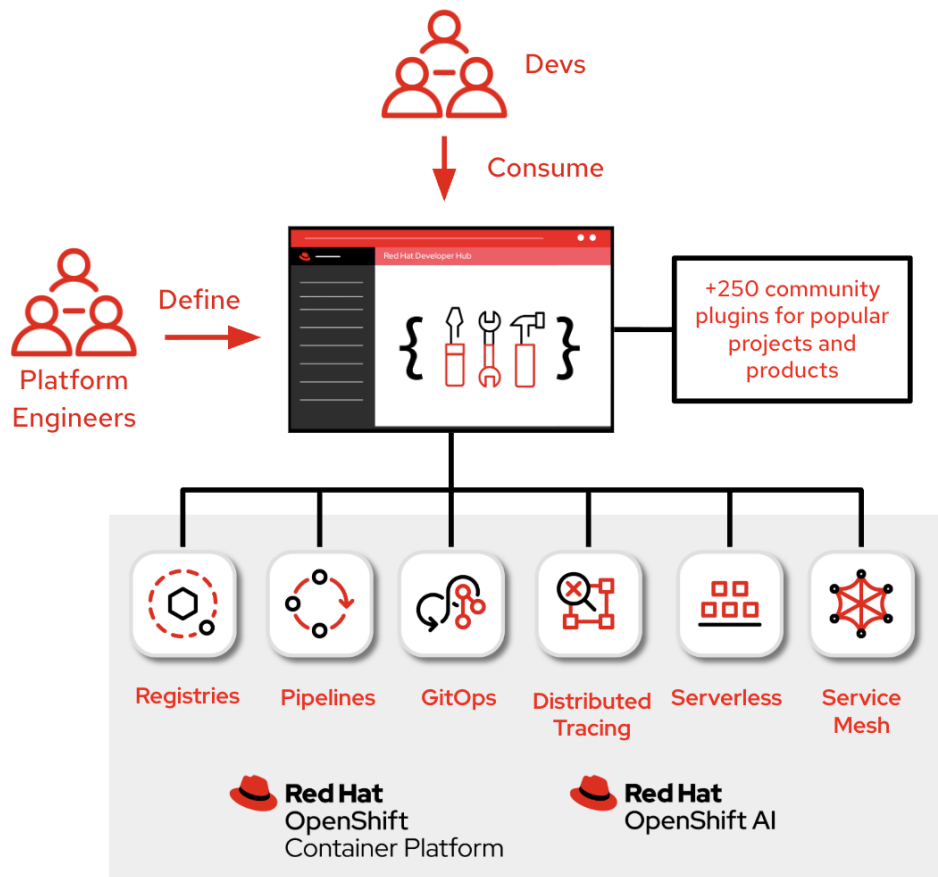
You explored how improving the **Developer Experience (DevEx)** enables organizations to reduce developer cognitive load, increase productivity, and accelerate software delivery.

This naturally raises an important question:

If a Software Factory consists of many different tools and platform capabilities, how do developers efficiently discover and use them without learning every underlying technology?

The answer is an **Internal Developer Portal**.

An Internal Developer Portal provides developers with a single, self-service interface for interacting with the Software Factory. Instead of navigating multiple applications for source control, CI/CD, deployment, documentation, observability, and development environments, developers can access everything from one centralized portal.



The illustration above demonstrates this concept. Platform Engineers build and maintain the Software Factory by integrating capabilities such as source control, CI/CD pipelines, GitOps, registries, service mesh, distributed tracing, and other platform services. Developers then consume these capabilities through a unified interface provided by the Internal Developer Portal.

What is Red Hat Developer Hub?

Red Hat Developer Hub (RHDH) is an enterprise-grade Internal Developer Portal built on the CNCF Backstage open source project. Rather than replacing the tools already used within the Software Factory, Developer Hub integrates them into a single developer experience.

Developer Hub provides a central location where developers can:

- Discover software components, APIs, services, and infrastructure resources.
- Create new applications using standardized Software Templates (Golden Paths).
- Launch cloud-based development environments.
- Monitor build pipelines and deployment status.
- Access project documentation.
- Navigate the complete application delivery lifecycle from a single interface.

This enables developers to focus on building applications instead of spending time locating tools, configuring environments, or learning platform-specific implementation details.

Developer Hub within the Software Factory

The Software Factory consists of multiple platform capabilities that work together to automate software delivery.

Developer Hub acts as the entry point that brings these capabilities together into a unified workflow.

Software Factory Capability	How Developer Hub Integrates
Source Control	Creates and links application repositories.
Development Environment	Launches cloud-based development workspaces using OpenShift Dev Spaces.
Continuous Integration	Displays pipeline execution and build status through Tekton integrations.
Continuous Delivery	Shows deployment status through GitOps integrations such as Argo CD.
Software Catalog	Provides a searchable inventory of software components, services, APIs, and ownership information.
Documentation	Makes project documentation available through TechDocs.
Observability	Integrates with monitoring and visualization platforms through plugins.

Rather than interacting with each platform independently, developers access these capabilities through a consistent and self-service experience.

Why is Developer Hub Important?

As organizations adopt microservices, Kubernetes, GitOps, and Platform Engineering practices, the number of tools involved in software delivery continues to grow.

Without a centralized developer portal, developers often need to:

- Navigate multiple web consoles.
- Search for project documentation.
- Locate repositories and deployment pipelines.

- Understand platform-specific implementation details.
- Request infrastructure from platform teams.

Developer Hub addresses these challenges by providing:

- **Discoverability** - Searchable software catalogs and documentation.
- **Self-service** - Automated Software Templates that generate complete application projects.
- **Standardization** - Every project follows organizational standards and best practices.
- **Developer productivity** - Developers spend less time configuring tools and more time writing code.

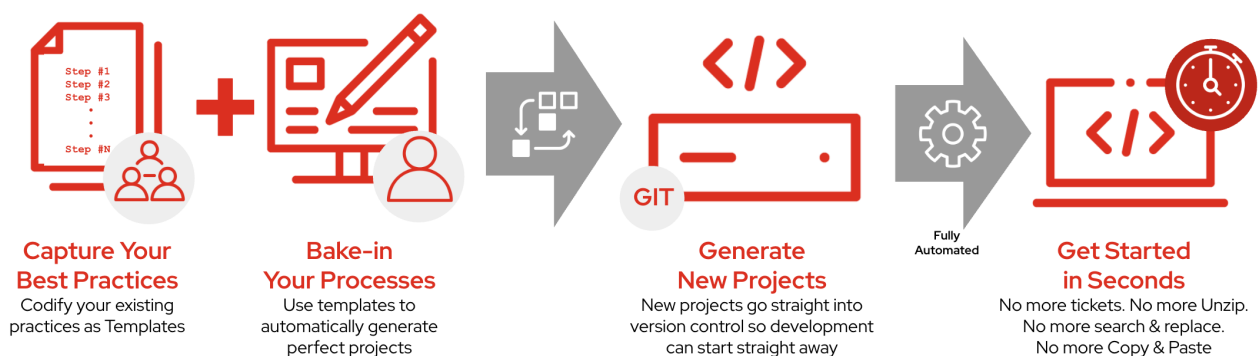
By reducing cognitive load and simplifying common development tasks, Developer Hub helps organizations improve both developer productivity and software delivery consistency.

Self-Service with Software Templates

Developers need a fast and consistent way to create **new** applications without manually configuring repositories, pipelines, deployment manifests, and platform integrations.

This is where **Software Templates** come in.

Software Templates, also known as **Golden Paths**, provide standardized project blueprints that automate the creation of new applications. Rather than starting from an empty repository, developers begin with an organization-approved template that already includes the required project structure, configurations, and automation.



The diagram above illustrates how Software Templates simplify application development by capturing organizational best practices and turning them into reusable automation.

Capture Your Best Practices

Platform Engineers work with development teams to identify proven architectural patterns, coding standards, deployment practices, and organizational policies.

Instead of documenting these practices for developers to follow manually, they are codified into reusable Software Templates.

As a result, every new project begins with the same approved foundation.

Bake Your Processes into Templates

Once best practices have been captured, Platform Engineers package them into Software Templates.

A template defines everything required to bootstrap a new application, including:

- Project structure and source code scaffold
- Git repository creation
- CI/CD pipeline configuration
- Deployment manifests, such as Helm charts or Kustomize overlays
- Dev Spaces configuration for cloud-based development
- Software Catalog registration
- Organizational standards and recommended configurations

Developers simply provide a few project-specific details, while the template automatically performs the remaining setup.

Generate New Projects Automatically

When a developer selects a Software Template from Red Hat Developer Hub, the platform automatically provisions the required resources.

Instead of manually creating repositories, configuring pipelines, or preparing deployment manifests, these tasks are performed automatically.

The generated project is immediately committed to the organization's version control system and is ready for development.

This allows developers to focus on implementing business functionality instead of performing repetitive setup tasks.

Get Started in Seconds

Traditional project onboarding often involves several manual activities, including:

- Requesting repositories
- Downloading starter projects
- Copying and modifying configuration files
- Creating CI/CD pipelines
- Configuring deployment resources

Software Templates eliminate these repetitive tasks by automating the entire project creation process.

Developers receive a production-ready project within seconds, while Platform Engineers can ensure every project follows organizational standards from the very beginning.

Key Benefits of Software Templates

Software Templates provide advantages for both developers and platform teams.

For developers:

- Quickly create new projects using proven architectural patterns.
- Eliminate repetitive project setup activities.
- Begin development immediately with a fully configured project.
- Follow organizational standards without additional effort.

For Platform Engineers:

- Standardize application architectures across teams.
- Embed security, compliance, and operational best practices into every project.
- Reduce support requests and onboarding time.
- Continuously improve templates as organizational standards evolve.



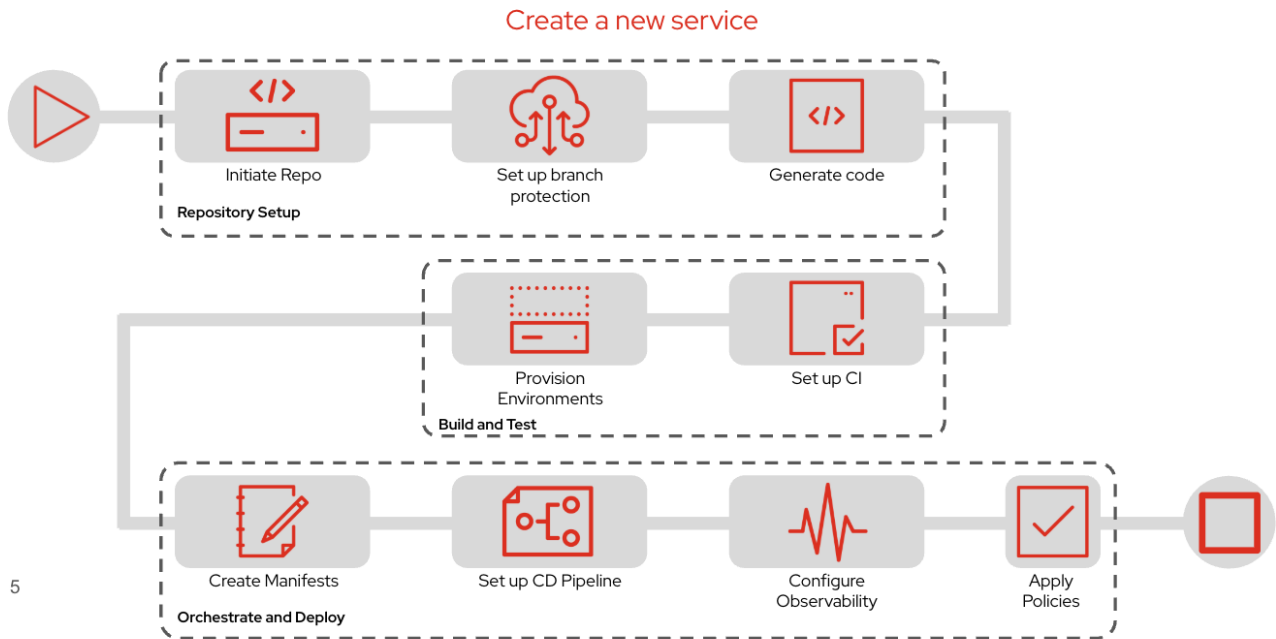
Software Templates do not replace developer creativity. Instead, they automate the repetitive foundation of every project, allowing developers to spend more time building business functionality while Platform Engineers ensure consistency across the Software Factory.

Software Template Workflow

In the previous section, you learned that Software Templates, also known as **Golden Paths**, automate the creation of new projects using organization-approved best practices.

From a developer's perspective, creating a new application is simple—they select a Software Template, provide a few project details, and click **Create**.

Behind the scenes, however, Red Hat Developer Hub orchestrates a series of automated tasks that prepare the project for development, deployment, and operations.



The workflow above illustrates how a Software Template automates the complete project bootstrap process.

Repository Setup

The first stage prepares the application's source repository.

Developer Hub automatically performs tasks such as:

- Creating a new Git repository.
- Configuring repository permissions.
- Applying branch protection policies.
- Generating the initial application source code from the selected template.

Instead of manually creating repositories and copying starter projects, developers receive a ready-to-use project that already follows organizational standards.

Build and Test

After the repository has been created, the template prepares everything required for continuous integration.

Typical automation includes:

- Provisioning development and testing environments.
- Creating Continuous Integration (CI) pipelines.
- Configuring build automation.
- Preparing the project for automated testing.

When developers commit their code, the generated pipeline is already available to build, test, and

validate the application.

Orchestrate and Deploy

The final stage prepares the application for deployment into Kubernetes.

Software Templates automatically generate deployment resources, including:

- Kubernetes deployment manifests.
- Helm charts or Kustomize overlays.
- Continuous Delivery (CD) pipeline configuration.
- Observability integrations for logs, metrics, and traces.
- Organizational security and governance policies.

By the time the template finishes executing, the application is ready to move through the Software Factory using standardized deployment workflows.

End-to-End Automation

Rather than automating only project creation, Software Templates establish the complete foundation for application delivery.

The generated project already includes everything needed to participate in the organization's Software Factory, including:

- Source control
- Continuous Integration
- Continuous Delivery
- Deployment manifests
- Observability
- Security and governance

As a result, developers can begin implementing business functionality immediately instead of spending time performing repetitive project setup activities.



Software Templates are created and maintained by Platform Engineers.

As organizational standards evolve, Platform Engineers update the template once, allowing every newly generated application to automatically inherit the latest best practices without requiring developers to manually reconfigure their projects.

Key Takeaway

A Software Template is much more than a project scaffold.

It automates the complete application onboarding process by provisioning repositories, pipelines,

deployment resources, and operational integrations, allowing developers to focus on writing application code while the Software Factory handles the surrounding platform configuration.

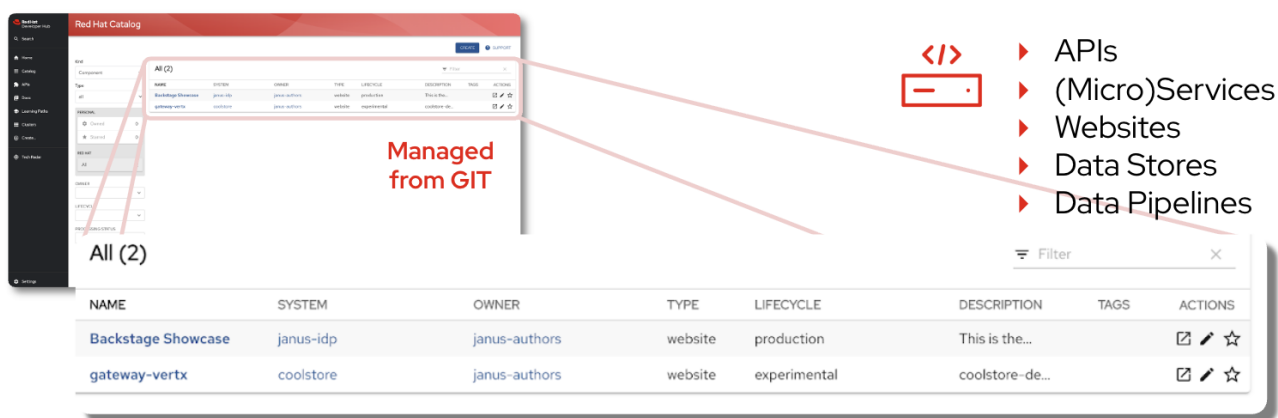
Software Catalog

As organizations adopt microservices and Platform Engineering, the number of software assets grows rapidly. Applications are no longer built as a single monolithic system. Instead, they consist of multiple services, APIs, databases, libraries, and supporting infrastructure owned by different development teams.

Without a centralized inventory, developers often struggle to answer questions such as:

- Which team owns this service?
- Where is the source code located?
- Which APIs does this application expose?
- Is this service still actively maintained?
- Where can I find the documentation?

The **Software Catalog** addresses these challenges by providing a centralized inventory of software assets across the organization.



The Software Catalog acts as a single source of truth for software components and their relationships. Rather than manually tracking applications in spreadsheets or documentation portals, Platform Engineers register software assets within the catalog, allowing developers to quickly discover the information they need.

What Does the Software Catalog Track?

The Software Catalog organizes software into different types of entities.

Entity Type	Description
Components	Individual software components such as microservices, websites, backend services, frontend applications, or shared libraries.

Entity Type	Description
APIs	REST, GraphQL, gRPC, or other APIs exposed by applications for communication with other services.
Resources	Infrastructure resources required by applications, such as databases, object storage, message brokers, or Kubernetes resources.
Systems	Collections of related components that together deliver a business application or platform.
Domains	Higher-level business or organizational groupings that contain multiple systems.
Users	Individual developers or engineers associated with software components.
Groups	Development teams or organizational groups responsible for owning and maintaining software.

More Than an Inventory

The Software Catalog is more than a list of applications.

Each catalog entry contains valuable metadata that helps developers understand and manage software throughout its lifecycle, including:

- Application owner
- Source code repository
- Lifecycle stage (Experimental, Development, Production, Deprecated)
- System relationships
- API dependencies
- Technical documentation
- CI/CD pipelines
- Deployment status

This information allows developers to quickly understand an application without searching across multiple tools.

Managed as Code

One of the key principles of Red Hat Developer Hub is that the Software Catalog is **managed as code**.

Each software component includes a `catalog-info.yaml` file in its source repository. This file defines the metadata that describes the component, including its name, owner, lifecycle, type, and relationships to other software assets.

Because the catalog is stored alongside the application's source code, changes to metadata follow the same Git-based workflows used for application development.

This approach ensures that catalog information remains version-controlled, reviewable, and automatically synchronized with the latest state of the application.

Benefits of the Software Catalog

The Software Catalog provides value to both developers and platform teams.

For developers:

- Quickly discover applications, APIs, and services.
- Identify application owners and responsible teams.
- Access repositories, documentation, and deployment information from one location.
- Understand relationships between software components.

For Platform Engineers:

- Maintain an accurate inventory of software assets.
- Improve governance and software ownership.
- Standardize metadata across development teams.
- Enable self-service discovery throughout the organization.



Think of the Software Catalog as the organization's **software inventory**.

Just as a library catalog helps readers quickly find books, the Software Catalog helps developers discover applications, services, APIs, documentation, ownership information, and platform resources from a single location.

Key Takeaway

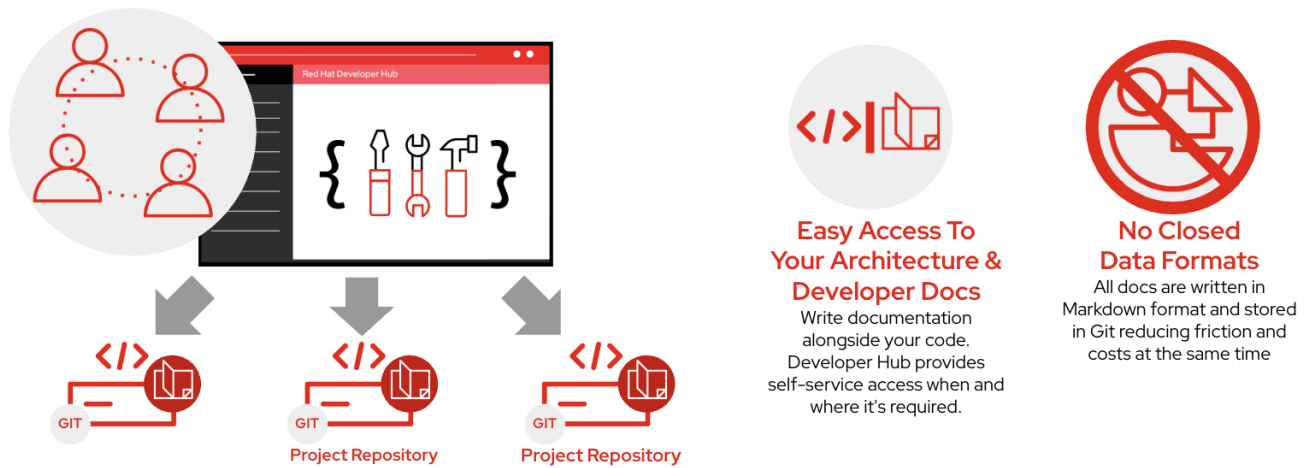
The Software Catalog serves as the central inventory of the Software Factory. It enables developers to discover software assets, understand ownership and relationships, and access the information they need without searching across multiple repositories and tools.

Documentation as Code with TechDocs

As software systems grow in complexity, keeping documentation accurate becomes just as important as maintaining the application itself. However, documentation is often stored separately from the source code—in shared drives, wikis, or knowledge bases—which can quickly become outdated as applications evolve.

To solve this problem, many organizations adopt the concept of **Documentation as Code**.

Documentation as Code treats documentation like application source code. Instead of maintaining documentation in external systems, it is written alongside the application's source code, version-controlled in Git, and updated using the same development workflow.



The illustration above shows how every project maintains its own documentation within its source repository. Red Hat Developer Hub automatically discovers this documentation and presents it through a centralized portal, allowing developers to easily search and access architecture guides, API documentation, deployment instructions, and operational runbooks.

Why Documentation as Code?

Keeping documentation close to the source code provides several advantages.

When developers update application code, they can update the documentation as part of the same change. Since both code and documentation follow the same Git workflow, every change is reviewed, version-controlled, and synchronized with the application's lifecycle.

This approach significantly reduces the likelihood of outdated or inconsistent documentation.

TechDocs in Red Hat Developer Hub

Red Hat Developer Hub includes **TechDocs**, a documentation platform built around the Documentation as Code principle.

Instead of storing documentation in a separate knowledge base, TechDocs automatically renders documentation directly from the application's Git repository.

Documentation is typically written using Markdown and organized alongside the application source code, making it easy for development teams to maintain and improve documentation throughout the software lifecycle.

Centralized Documentation Experience

Although documentation remains distributed across individual repositories, Developer Hub provides a centralized experience for consuming it.

Developers can quickly discover documentation for applications, APIs, libraries, and platform components without needing to search across multiple repositories.

Typical documentation includes:

- Application architecture
- API documentation
- Deployment guides
- Development setup instructions
- Operational runbooks
- Troubleshooting guides
- Architecture Decision Records (ADRs)

This enables every software component in the Software Catalog to have its own version-controlled documentation that is immediately available through Developer Hub.

Open Standards

TechDocs uses open and widely adopted technologies.

Documentation is typically:

- Written in Markdown.
- Stored alongside application source code in Git.
- Version-controlled with the application.
- Reviewed through the same pull request workflow as application code.

Because documentation is stored in open formats, organizations are not locked into proprietary documentation platforms or closed file formats.

Benefits of Documentation as Code

For developers:

- Access project documentation directly from Developer Hub.
- Keep documentation synchronized with application code.
- Collaborate using familiar Git workflows.
- Search documentation across multiple projects from a single portal.

For Platform Engineers:

- Encourage consistent documentation standards.
- Reduce outdated documentation.
- Improve knowledge sharing across teams.
- Eliminate dependence on proprietary documentation systems.



Think of TechDocs as the documentation layer of the Software Factory.

Just as the Software Catalog helps developers discover software components,

TechDocs helps them discover the knowledge required to build, operate, and maintain those components.

Key Takeaway

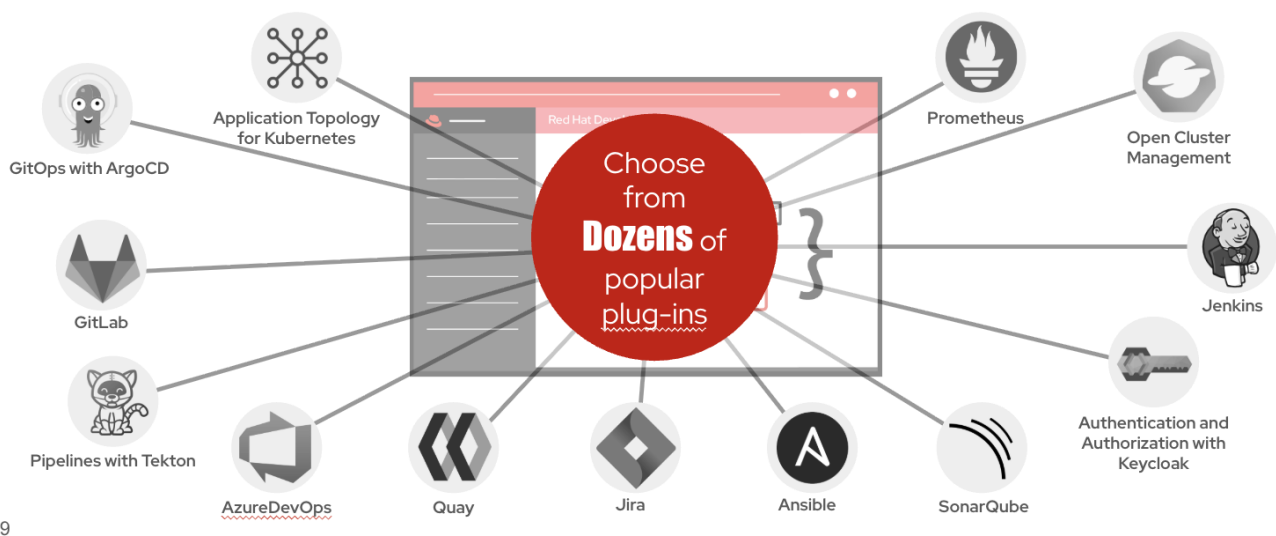
TechDocs extends Red Hat Developer Hub with Documentation as Code capabilities. By storing documentation alongside application source code and rendering it centrally through Developer Hub, organizations ensure that documentation remains accurate, searchable, version-controlled, and available throughout the software development lifecycle.

Extending Developer Hub with Plugins

So far, you've learned how Red Hat Developer Hub helps developers discover software through the Software Catalog, create new applications using Software Templates, and access project documentation with TechDocs.

These capabilities provide the foundation of an Internal Developer Portal. However, modern software delivery involves many additional tools that developers use throughout the application lifecycle.

Rather than requiring developers to switch between multiple consoles, Red Hat Developer Hub integrates these tools directly into the developer experience through **Plugins**.



9

The illustration above shows how Developer Hub serves as a central interface that integrates with the tools commonly found in a Software Factory. Instead of replacing these tools, Developer Hub connects to them and presents their information through a unified, self-service experience.

What are Plugins?

Plugins extend the capabilities of Red Hat Developer Hub by integrating external platforms and services into the portal.

Each plugin connects Developer Hub with a specific tool, allowing developers to view information and perform common tasks without leaving the portal.

Depending on organizational requirements, Platform Engineers can enable only the plugins that are relevant to their Software Factory.

Common Plugin Integrations

Developer Hub includes plugins for many popular software development and platform technologies.

Plugin	Purpose
GitLab or GitHub	Access source repositories, merge requests, and project information.
OpenShift Dev Spaces	Launch cloud-based development workspaces directly from Developer Hub.
Tekton Pipelines	Monitor Continuous Integration (CI) pipeline execution and build status.
Argo CD	View GitOps deployment status and application synchronization.
Quay	Access container images stored in the organization's image registry.
Prometheus	View application metrics and monitoring information.
SonarQube	Review code quality, security issues, and technical debt.
Keycloak	Provide centralized authentication and role-based access control.
Jira	Link software components to user stories, defects, and project work.
Ansible	Automate infrastructure provisioning and operational tasks.
Open Cluster Management	Monitor and manage applications deployed across multiple OpenShift clusters.

A Unified Developer Experience

Without Developer Hub, developers often switch between multiple applications throughout a typical workday.

For example, a developer may need to:

- Access source code in Git.
- Check pipeline status in Tekton.
- Review deployments in Argo CD.
- View monitoring dashboards in Prometheus.
- Search documentation.
- Open Jira to track work items.

Although each tool serves a specific purpose, constantly switching between them interrupts developer productivity and increases cognitive load.

Developer Hub brings these capabilities together into a single interface, allowing developers to access the information they need without navigating multiple consoles.

Built for Extensibility

One of the strengths of Red Hat Developer Hub is its extensible architecture.

Because it is built on the Backstage open source framework, organizations can:

- Enable community-supported plugins.
- Develop custom plugins for internal tools.
- Integrate proprietary enterprise platforms.
- Customize the developer experience to match organizational workflows.

This flexibility allows organizations to evolve their Internal Developer Portal as their Software Factory grows.



Plugins do not replace the underlying tools.

Git remains your source control system, Tekton continues to execute CI pipelines, Argo CD manages GitOps deployments, and Prometheus collects monitoring data.

Developer Hub simply brings these capabilities together into a single developer experience.

Key Takeaway

Plugins transform Red Hat Developer Hub from a software catalog into a unified developer portal.

By integrating source control, development environments, CI/CD, GitOps, observability, security, and project management tools, Developer Hub becomes the single entry point through which developers interact with the entire Software Factory.

MCP in Developer Hub

AI Integration with Model Context Protocol (MCP)

As organizations adopt AI-powered developer assistants, Large Language Models (LLMs) need secure access to enterprise knowledge and platform capabilities.

The **Model Context Protocol (MCP)** provides a standardized way for AI applications, also known as **MCP clients**, to communicate with external systems through **MCP servers**.

Instead of requiring every AI assistant to implement custom integrations for different tools, MCP defines a common protocol for discovering and invoking capabilities exposed by external systems.

[mcp overview] | [mcp-overview.png](#)

An MCP server exposes a collection of **tools**, where each tool performs a specific action against an external system. These tools can retrieve information, query APIs, access documentation, or interact with platform services.

Examples include:

- Querying software catalogs
- Retrieving documentation
- Searching source code repositories
- Accessing monitoring systems
- Performing web searches



MCP standardizes how AI assistants access external systems. It does not replace those systems.

MCP in Red Hat Developer Hub

Red Hat Developer Hub includes support for MCP through the Backstage MCP plug-in.

Unlike traditional MCP implementations that require building a separate MCP server, Developer Hub allows Platform Engineers to expose MCP tools by implementing simple Backstage backend plug-ins.

This approach enables organizations to make platform knowledge available to AI assistants without introducing additional infrastructure.

[rhdh mcp architecture] | [rhdh-mcp-architecture.png](#)

MCP Tools in Developer Hub

As an initial implementation, Red Hat provides two read-only MCP tool plug-ins.

Software Catalog MCP Tool

Allows AI assistants to query the complete Software Catalog, including component metadata such as ownership, lifecycle, relationships, repositories, and dependencies.

TechDocs MCP Tool

Allows AI assistants to retrieve documentation published through TechDocs, enabling developers to ask natural language questions about architecture, APIs, runbooks, and operational procedures.

These tools are intentionally read-only, allowing AI assistants to retrieve information without modifying platform resources.



Write operations require additional security controls and are typically introduced only after appropriate authentication, authorization, and governance mechanisms are in place.

Why MCP Matters

MCP helps extend the Developer Experience by allowing AI assistants to retrieve platform knowledge from trusted sources.

Instead of manually searching documentation or navigating multiple tools, developers can ask questions such as:

- "Who owns this service?"
- "Show me the documentation for this API."
- "Which Git repository contains this application?"
- "What is the lifecycle status of this component?"

The AI assistant retrieves this information directly from Developer Hub using MCP, providing accurate and up-to-date responses.



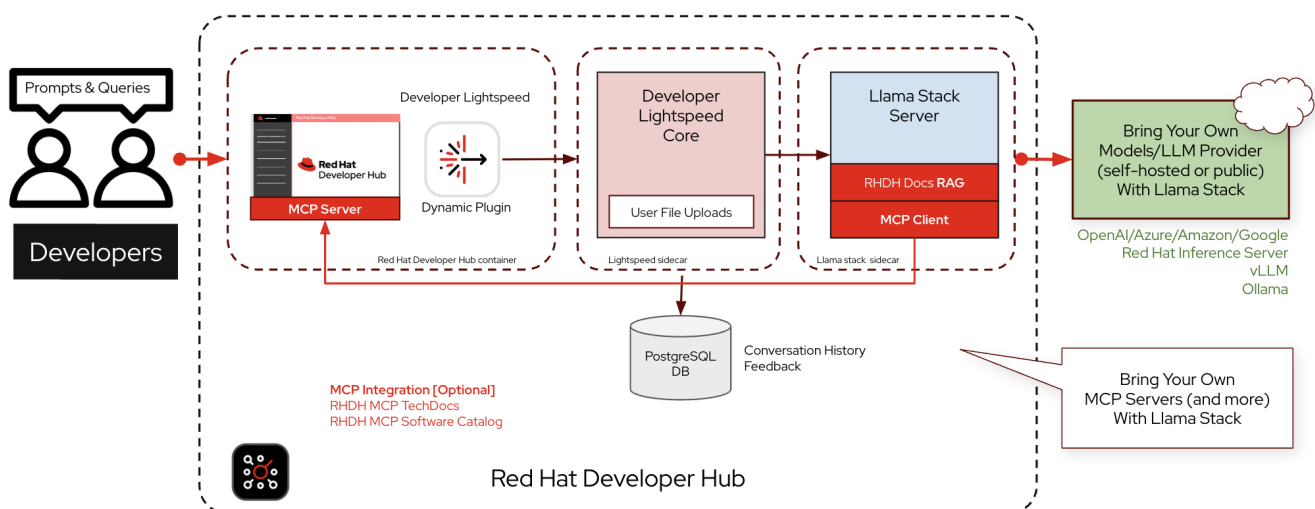
Think of MCP as a standardized bridge between AI assistants and the Software Factory. It enables AI to retrieve trusted platform information while allowing organizations to maintain control over what information is exposed.

Developer Lightspeed

AI-Powered Developer Assistance with Developer Lightspeed

As modern software platforms continue to grow in complexity, developers spend a significant amount of time performing tasks beyond writing code. These tasks include searching documentation, understanding platform capabilities, troubleshooting issues, and learning new tools.

Red Hat Developer Lightspeed helps reduce this overhead by providing an AI-powered assistant directly within Red Hat Developer Hub.



Developer Lightspeed integrates into the existing Developer Hub experience through a plug-in,

allowing developers to ask natural language questions without leaving the portal.

Examples include:

- How do I deploy my application using GitOps?
- Show me the documentation for this component.
- Who owns this service?
- How do I create a new software template?

Instead of manually searching multiple documentation sites or repositories, developers can interact with the platform using conversational prompts.

How Developer Lightspeed Works

When a developer submits a prompt through Developer Hub, the request is processed by the Developer Lightspeed service.

The service acts as an intermediary between Developer Hub and the configured Large Language Model (LLM). It is responsible for:

- Authenticating requests
- Retrieving additional context
- Communicating with the configured AI model
- Returning context-aware responses to the developer

Organizations have the flexibility to use either self-hosted or public AI models, depending on their operational and security requirements.

Supported providers include:

- OpenAI
- Microsoft Azure OpenAI
- Google
- Amazon Bedrock
- Red Hat AI Inference Server
- vLLM
- Ollama



Developer Lightspeed is model-agnostic. Organizations can choose the AI model that best meets their deployment, compliance, and performance requirements.

Improving Response Accuracy with Retrieval-Augmented Generation (RAG)

Large Language Models provide general knowledge, but they are typically unaware of organization-specific platform information.

To improve the quality of responses, Developer Lightspeed uses a **Retrieval-Augmented Generation (RAG)** architecture.

Rather than relying solely on the model's training data, the service retrieves relevant information from trusted sources before generating a response.

For Red Hat Developer Hub, these sources include:

- Official Developer Hub documentation
- Software Catalog metadata
- TechDocs documentation (through MCP)

This allows the AI assistant to provide responses that are more accurate, relevant, and specific to the developer's environment.

Extending Platform Knowledge with MCP

Developer Lightspeed can also use the **Model Context Protocol (MCP)** to retrieve information from trusted platform services.

Developer Hub exposes MCP tools that allow the AI assistant to access:

- Software Catalog metadata
- TechDocs documentation

Instead of relying solely on the language model's knowledge, the AI assistant retrieves current platform information directly from Developer Hub, resulting in more reliable and context-aware responses.



Think of MCP as a secure bridge that allows AI assistants to access trusted platform information rather than relying only on what the language model already knows.

Conversation History

Developer Lightspeed can optionally persist conversations using PostgreSQL.

This enables organizations to:

- Preserve conversation history
- Collect user feedback
- Review interactions for compliance or auditing
- Analyze usage patterns to improve the developer experience

By default, conversation history can also be stored in memory for simpler deployments.

Key Takeaway

Developer Lightspeed enhances the Developer Experience by bringing AI-powered assistance

directly into Red Hat Developer Hub.

Rather than replacing existing platform tools, it helps developers interact with them more efficiently by:

- Answering platform-related questions
- Retrieving trusted documentation
- Accessing Software Catalog information through MCP
- Providing context-aware guidance using Retrieval-Augmented Generation (RAG)

This enables developers to spend less time searching for information and more time building applications.

Hands-on Lab

Install and configure Red Hat Developer Hub

After the Software Factory work in Lab 1, you need a single front door where developers discover services, templates, and links to the tools they use every day. Red Hat Developer Hub provides that experience on OpenShift when you pair the Red Hat Developer Hub Operator with a small `app-config` and a `Backstage` custom resource.

Red Hat Developer Hub is a component of Red Hat Advanced Developer Suite that enables platform engineers to build internal developer portals for their organization. It is based on the `Backstage` project for building internal developer portals. This project was donated to the `CNCF` by Spotify in 2020. Platform engineers can use Red Hat Developer Hub to build internal developer portals. Doing so involves integrating Red Hat Developer Hub with various data sources, cataloging existing software components, infrastructure, and resources, configuring authentication, and more.

Learning objectives

By the end of this lab, you will be able to:

- Install the Red Hat Developer Hub Operator on your cluster.
- Create a project, supply a minimal `app-config` through a ConfigMap, and deploy a `Backstage` custom resource.
- Verify pods, routes, and sign-in to Red Hat Developer Hub using the guest provider configured for lab use.
- Explain why guest authentication is restricted to non-production scenarios.

Exercise 1: Install the Operator and create the `setup-rhdh` project

1. Open the OpenShift web console at `{openshift_console_url}` and sign in with `{user}` and `{password}`.
2. In the OpenShift Web Console, go to menu:Ecosystem[Software Catalog]
3. Search for **Red Hat Developer Hub**.



Make sure that you choose the **Operator Backed** entry from the results.

4. Click btn:[Install]
5. Keep the operator installation namespace as `rhdh-operator`
6. Accept the defaults for the rest (All namespaces, Automatic approval) and click btn:[Install]
7. Wait for the operator to reach the `Succeeded` phase

From the terminal tab, create the `setup-rhdh` the project:

```
oc create namespace setup-rhdh
```

Expected output (namespace line may vary slightly by cluster):

```
Now using project "setup-rhdh" on server "https://api.openshift.example.com:6443".
```

Exercise 2: Create the **rhdh-config** ConfigMap

Red Hat Developer Hub reads **app-config** settings from files mounted into the application container. You store that file in a ConfigMap in the **setup-rhdh** namespace.

1. In the console, open **Workloads** and **ConfigMaps** while **setup-rhdh** is selected.
2. Click **Create ConfigMap**, switch to **YAML view**, replace the placeholder content, and apply the following manifest.

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: rhdh-config
  namespace: setup-rhdh
data:
  app-config.yaml: |
    app:
      title: Red Hat Developer Hub
    backend:
      reading:
        allow:
          - host: 'gitlab.{openshift_cluster_ingress_domain}'
    auth:
      providers:
        guest:
          dangerouslyAllowOutsideDevelopment: true
```

The **backend.reading.allow** entry permits the Developer Hub backend to fetch catalog data and templates from `{gitlab_url}`.



Guest access with **dangerouslyAllowOutsideDevelopment** is only for isolated lab environments. Do not use this pattern when the cluster reaches real data sources or production systems.

3. Click **Create** to save the ConfigMap.

Verify

```
oc get configmap rhdh-config -n setup-rhdh -o yaml
```

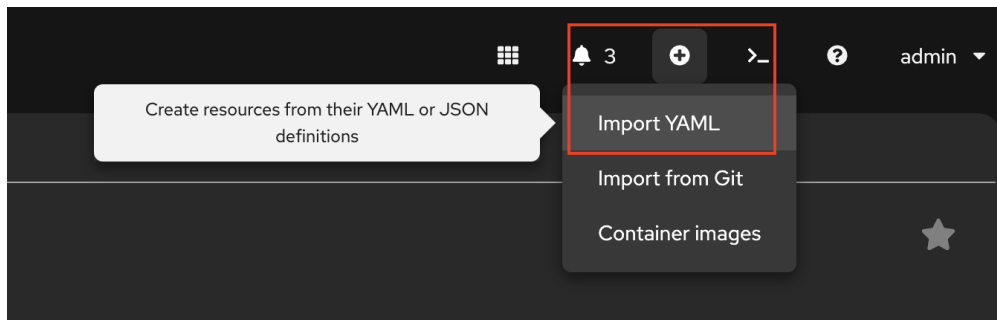
Expected output (truncated):

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: rhdh-config
  namespace: setup-rhdh
...
```

Exercise 3: Deploy the **Backstage** custom resource

The Red Hat Developer Hub Operator watches for **Backstage** resources and reconciles PostgreSQL and application pods, routes, and mounts for your ConfigMap. The `spec.application.appConfig` section mounts **rhdh-config** into the Backstage container so the running instance loads `app-config.yaml`, including `backend.reading.allow` for the GitLab host at `{gitlab_url}`.

1. Click the plus (+) icon in the top bar and choose **Import YAML**.



2. Paste the following manifest and select your **setup-rhdh** project if prompted.

```
apiVersion: rhdh.redhat.com/v1alpha5
kind: Backstage
metadata:
  name: rhdh
  namespace: setup-rhdh
spec:
  application:
    appConfig:
      mountPath: /opt/app-root/src
      configMaps:
        - name: rhdh-config
    route:
      enabled: true
  database:
    enableLocalDb: true
```

3. Click **Create**.

Verify

Watch the pods until PostgreSQL and the main Backstage pod report **Running** and ready:

```
oc get pods -n setup-rhdh
```

Expected output (names and ages will differ):

NAME	READY	STATUS	RESTARTS	AGE
backstage-rhdh-xxxxxxxx-xx	1/1	Running	0	2m
postgresql-xxxxxxxx-xx	1/1	Running	0	3m

Check the custom resource and route:

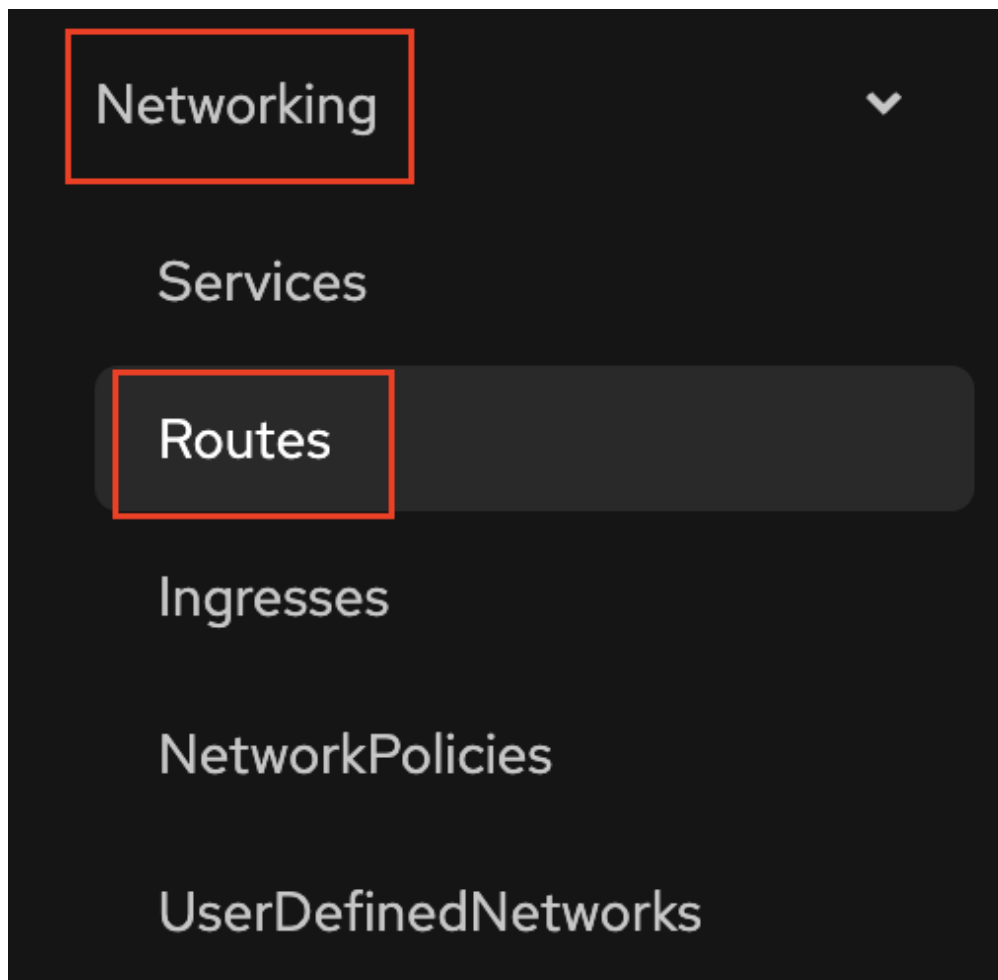
```
oc get backstage -n setup-rhdh
```

```
oc get route backstage-rhdh -n setup-rhdh -o jsonpath='{https://{.spec.host}{"\n"}'}
```

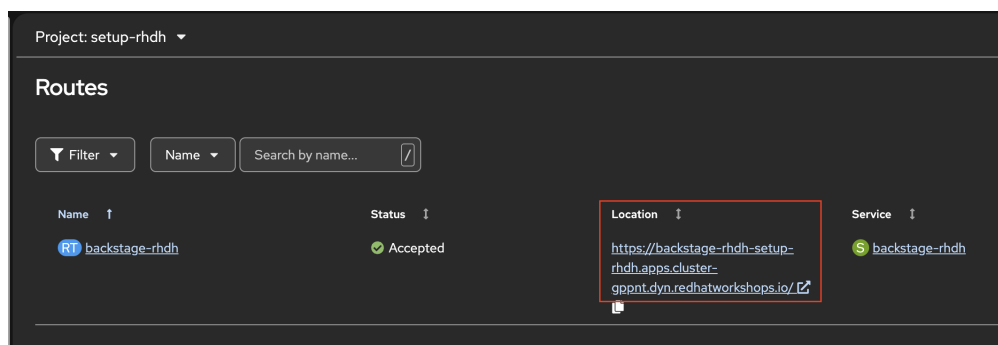
Expected route list includes an entry for the Developer Hub host with port 443.

Exercise 4: Open Red Hat Developer Hub and sign in

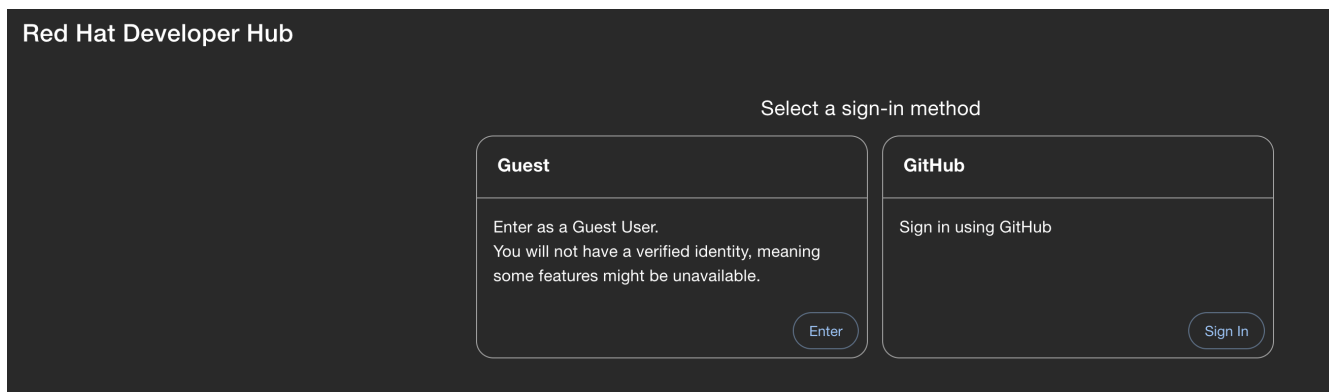
1. Open **Networking** and **Routes** for project **setup-rhdh**.



2. Click the **Location** URL for the Developer Hub route, or use the **Developer Hub** tab in the Showroom UI if it is configured for your environment.



If you see **Application is not available**, wait one minute and refresh. Pods may still be completing readiness checks. . On the sign-in screen, choose **Guest**. You should land on the home page signed in as a guest user.



Verify

Confirm you can load the UI and that the browser shows Red Hat Developer Hub with **Guest** context.

Troubleshooting

Issue: Pods stay in `CrashLoopBackOff` for the Backstage workload. **Solution:** List pods with `oc get pods -n setup-rhdh`, run `oc describe pod <pod-name> -n setup-rhdh` on the Backstage pod, and confirm the ConfigMap mounted without typos. Ensure `app-config` URLs match your route hostname.

Issue: Route returns 503 or application not available. **Solution:** Wait for readiness probes to pass. Check `oc get pods -n setup-rhdh` until both pods are ready.

Learning outcomes checkpoint

Before you leave this lab, confirm you can:

- ☐ Explain how a portal complements the Software Factory capabilities from Lab 1.
- ☐ List the main resources you created (`Project`, `ConfigMap`, `Backstage` CR) and what each one does.
- ☐ Contrast guest access with enterprise authentication for real environments.

Summary

You deployed Red Hat Developer Hub using the Operator, a minimal `app-config` in a ConfigMap, and a `Backstage` custom resource in the `setup-rhdh` namespace.

Next steps

Connect Red Hat Developer Hub to Git providers, identity providers, and GitOps so catalogs and templates reflect your toolchain. See [Red Hat Developer Hub documentation](#) for supported integrations and security guidance.

For Golden Path templates, Dev Spaces integration, and authentication configuration, continue with [Developer Hub: templates and integrations](#).

Developer Hub: templates and integrations

Complete [Install and configure Red Hat Developer Hub](#) before this section so you have a running instance.

Overview

In this section you will register a Golden Path or Software template that scaffolds new Quarkus projects with CI/CD pre-configured, integrate Dev Spaces for one-click workspace creation, and configure OpenShift OAuth for enterprise authentication.

Software templates in Red Hat Developer Hub enable your team(s) to create Entities, such as new Components, and - through the use of "actions" provided by plugins - create resources in other systems such as your GitLab, GitHub, and OpenShift GitOps instances. Templates themselves are represented as Entities in the Software Catalog, meaning you can import them similar to any other Entity!

Platform Engineers will often be the authors of Templates, and use them to create "golden paths" that follow best-practices and use approved processes and tooling. Development teams will be the consumers of Templates to create new software and automate their tasks. Using Templates reduces cognitive load on the development teams by allowing them to focus on development tasks, while platform concerns are addressed by the template.

Templates are defined using YAML, but are rendered as a rich form in the Red Hat Developer Hub UI when used by development teams.

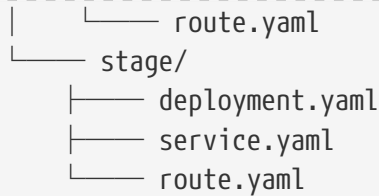
Register a Golden Path Template

Golden Path templates provide a self-service way for developers to create new projects that follow organizational standards.

Template Structure

A Golden Path template consists of:

```
golden-path-quarkus/
├── template.yaml           # Template definition
├── skeleton/
│   ├── devfile.yaml       # Dev Spaces workspace config
│   ├── pom.xml            # Quarkus project with dependencies
│   ├── src/               # Application source code
│   ├── catalog-info.yaml  # Backstage catalog entity
│   ├── tekton/            # Pipeline definitions
│   │   ├── pipeline.yaml
│   │   └── trigger.yaml
│   └── gitops/            # Argo CD manifests
│       ├── dev/
│       │   ├── deployment.yaml
│       │   └── service.yaml
```



Create the Template Definition

Create a Git repository in GitLab to hold the template definition.:

1. Log into GitLab:

```
{gitlab_url}
```

2. Navigate to the Software Factory group
3. Create a new empty and public project: **golden-path-quarkus**
4. Uncheck the "Initialize repository with a README" checkbox.
5. Clone it locally:

```
cd ~/workshop
git clone {gitlab_url}/software-factory/golden-path-quarkus.git
cd golden-path-quarkus
```

Create **template.yaml** in your Git repository:

```
apiVersion: scaffolder.backstage.io/v1beta3
kind: Template
metadata:
  name: quarkus-golden-path
  title: Quarkus Application (Golden Path)
  description: |
    Create a Quarkus application with CI/CD pipeline, GitOps deployment,
    Dev Spaces workspace, and OpenTelemetry instrumentation pre-configured.
  tags:
    - quarkus
    - java
    - recommended
spec:
  owner: platform-team
  type: service
  parameters:
    - title: Application Details
      required:
        - name
        - owner
  properties:
```

```

    name:
      title: Application Name
      type: string
      pattern: "^[a-z0-9-]+$"
    description:
      title: Description
      type: string
    owner:
      title: Owner
      type: string
      ui:field: OwnerPicker

- title: Repository
  required:
    - repoUrl
  properties:
    repoUrl:
      title: Repository Location
      type: string
      ui:field: RepoUrlPicker
      ui:options:
        allowedHosts:
          - {gitlab_url}
  steps:
    - id: fetch-skeleton
      name: Fetch Skeleton
      action: fetch:template
      input:
        url: ./skeleton
        values:
          name: ${ parameters.name }
          description: ${ parameters.description }
          owner: ${ parameters.owner }

    - id: publish
      name: Publish to GitLab
      action: publish:gitlab
      input:
        repoUrl: ${ parameters.repoUrl }
        description: ${ parameters.description }
        defaultBranch: main

    - id: register
      name: Register in Catalog
      action: catalog:register
      input:
        repoContentsUrl: ${ steps['publish'].output.repoContentsUrl }
        catalogInfoPath: /catalog-info.yaml

  output:
    links:

```

```
- title: Repository
  url: ${ steps['publish'].output.remoteUrl }
- title: Open in Catalog
  icon: catalog
  entityRef: ${ steps['register'].output.entityRef }
```

Create the Catalog Entity

Add `catalog-info.yaml` to the skeleton directory:

```
apiVersion: backstage.io/v1alpha1
kind: Component
metadata:
  name: ${ values.name }
  description: ${ values.description }
  annotations:
    backstage.io/techdocs-ref: dir:.
    argocd/app-name: ${ values.name }-dev
    janus-idp.io/tekton: ${ values.name }
spec:
  type: service
  lifecycle: production
  owner: ${ values.owner }
```

Commit and push the template repository:

```
git add .
git commit -m "feat: initial commit"
git push origin main
```

Register the Template in Developer Hub

Red Hat Developer Hub discovers Software Templates through **catalog locations** in `app-config.yaml`. After you push `template.yaml` to GitLab, add a `catalog` section that points at that file so the catalog importer loads your Golden Path template. You update the existing `rhdh-config` ConfigMap in the `setup-rhdh` namespace—the same ConfigMap created in [Install and configure Red Hat Developer Hub](#).

1. Open `{openshift_console_url}[^]` and switch to the **setup-rhdh** project.
2. In the console, open **Workloads** → **ConfigMaps** and select **rhdh-config**.
3. Click **Actions** → **Edit ConfigMap** and stay in **YAML view** (or open the **Data** tab and edit the `app-config.yaml` key).
4. Under `data.app-config.yaml`, keep your existing `app` and `auth` settings and add the `catalog` block below. Do not replace the whole file—merge the new section with what is already there.

```
catalog:
```

```
locations:
  - type: url
    target: '{gitlab_url}/software-factory/golden-path-quarkus/-
      /raw/main/template.yaml'
    rules:
      - allow: [Template]
```

5. Click **Save** to apply the ConfigMap.

The Red Hat Developer Hub Operator rolls out the change and restarts the application pod. When the pod is **Running** again, open Developer Hub and confirm the template appears under menu:Create[Templates].

[\[Quarkus Golden Path template in Red Hat Developer Hub\]](#) | *rhdh-quarkus-golden-path-template.png*

Integrate with Dev Spaces

Add a Dev Spaces link to catalog components so developers can open a workspace directly from the component page.

Add the Dev Spaces URL Annotation

In the `catalog-info.yaml` skeleton, add the Dev Spaces annotation:

```
metadata:
  annotations:
    che.eclipse.org/che-server: {devspaces_url}
```



Do not forget to commit and push the changes to the Git repository.

```
git add catalog-info.yaml
git commit -m "feat: add dev spaces annotation"
git push origin main
```

When a component has this annotation, the Developer Hub UI shows a "Dev Spaces" link that opens a workspace for the component's repository.

Configure the Dev Spaces Plugin

Add the Dev Spaces plugin configuration to **rhdh-config** ConfigMap. Under `data.app-config.yaml`, keep your existing `app`, `auth` and other settings and add the `devSpaces` block below. Do not replace the whole file—merge the new section with what is already there.

```
devSpaces:
  defaultUrl: {devspaces_url}
```

Update the ConfigMap and wait for the pod to restart.

Configure OpenShift OAuth Authentication

Replace guest authentication with OpenShift OAuth for production-like access.

Create an OAuth Client

```
oc create -f - <<EOF
apiVersion: oauth.openshift.io/v1
kind: OAuthClient
metadata:
  name: rhdh-oauth-client
grantMethod: auto
redirectURIs:
  - https://backstage-rhdh-setup-
    rhdh.{openshift_cluster_ingress_domain}/api/auth/oidc/handler/frame
secret: GENERATE_A_RANDOM_SECRET
EOF
```



You can generate a random secret value using the following command:

```
node -p 'require("crypto").randomBytes(24).toString("base64")'
```

Create the Auth Secret

```
oc create secret generic rhdh-auth-secret -n setup-rhdh \
  --from-literal=AUTH_OIDC_CLIENT_ID=rhdh-oauth-client \
  --from-literal=AUTH_OIDC_CLIENT_SECRET=GENERATE_A_RANDOM_SECRET \
  --from-literal=AUTH_OIDC_METADATA_URL=https://oauth
  -openshift.{openshift_cluster_ingress_domain}/.well-known/openid-configuration
```

Update app-config

Replace the guest auth block with OIDC:

```
signInPage: oidc
auth:
  session:
    secret: MY_SESSION_SECRET
  environment: production
  providers:
    oidc:
      production:
        clientId: ${AUTH_OIDC_CLIENT_ID}
        clientSecret: ${AUTH_OIDC_CLIENT_SECRET}
```

```
metadataUrl: ${AUTH_OIDC_METADATA_URL}
prompt: auto
```



You can replace `MY_SESSION_SECRET` with a random value using the following command:

```
node -p 'require("crypto").randomBytes(24).toString("base64")'
```

Click **Save** and wait for the Backstage pod to restart.

`signInPage: oidc` shows the OpenShift sign-in flow on the login page. The `${AUTH_OIDC_*}` placeholders are resolved from the Secret mounted on the `Backstage` CR in the next step.

Mount the Secret in the Backstage CR

Mount `rhdh-auth-secret` on the `Backstage` custom resource so the OIDC client ID, secret, and metadata URL are available to `app-config` as environment variables.

1. Open `{openshift_console_url}[^]` and switch to the **setup-rhdh** project.
2. Go to **Ecosystem** → **Installed Operators**.
3. Select **Red Hat Developer Hub**.
4. Open the **Backstage** tab.
5. Click the **rhdh** instance created.
6. Click **Actions** → **Edit Backstage** and switch to **YAML view**.
7. Under `spec.application`, add the `extraEnvs` block below. Keep the existing `appConfig` and `route` sections from the initial deployment.

```
extraEnvs:
  secrets:
    - name: rhdh-auth-secret
```

8. Click **Save**.

The Red Hat Developer Hub Operator reconciles the CR and restarts the Backstage pod.

Verify OpenShift sign-in

1. Wait until the Backstage pod in **setup-rhdh** is **Running**.
2. Open the Developer Hub route URL from the first step in this section.
3. On the login page, choose **OIDC**.
4. Sign in with your OpenShift credentials (for example `{user}` / `{password}`).

You should land on the Developer Hub home page authenticated as your OpenShift user, not as

Guest.



The signed-in user must exist in the Software Catalog (or be mapped via your org setup) to use some catalog features. For template creation in this lab, guest access is no longer available after this change.

Test the Golden Path

1. Open Developer Hub and go to menu:Create[Templates]
2. Select the **Quarkus Application (Golden Path)** template
3. Fill in the parameters (name, owner, repository location)
4. Click btn:[Create]
5. Verify the template:
 - A new GitLab repository is created with the skeleton code
 - The component is registered in the Developer Hub catalog
 - The repository includes a devfile, Tekton pipeline, and GitOps manifests
6. Open the component in the catalog and click the Dev Spaces link to launch a workspace

Summary

Developer Hub now provides:

- **Golden Path templates** that scaffold complete projects with CI/CD, GitOps, and Dev Spaces pre-configured
- **Software Catalog** with component metadata, ownership, and links to pipelines and deployments
- **Dev Spaces integration** for one-click workspace creation from catalog components
- **OpenShift OAuth** for enterprise authentication

Next Steps

Proceed to [Day 2 Recap](#).